

项目概述

这段代码使用Go语言实现了一个线性回归可视化工具，通过梯度下降算法拟合线性模型，并实时展示拟合过程。主要功能包括：

- 生成带有噪声的样本数据点，模拟真实世界中的线性关系
- 使用梯度下降算法优化线性回归模型参数（权重 w 和偏置 b ）
- 实时可视化展示拟合过程，包括样本点、真实直线和拟合直线
- 显示训练进度、损失值和模型参数的变化

💡 该项目非常适合理解线性回归和梯度下降的工作原理，通过可视化可以直观地看到模型如何逐步逼近真实数据分布。

代码结构

主要组成部分

常量定义

屏幕尺寸、坐标轴参数、更新间隔等配置

全局变量

数据存储、模型参数、字体等全局状态

数据生成函数

生成符合线性关系的样本数据

模型训练函数

计算损失、梯度下降更新参数

可视化函数

绘制网格、坐标轴、数据点、直线等

游戏循环

Ebiten框架的Update和Draw方法，控制更新和渲染

程序执行流程

- 初始化：加载字体、设置参数、生成样本数据
- 游戏循环开始：
 - Update(): 按固定间隔执行梯度下降步骤
 - Draw(): 绘制当前状态的所有元素
- 训练完成：模型参数收敛，拟合直线接近真实直线

核心算法

线性回归模型

线性回归模型表示为：

$$y = w \cdot x + b$$

其中：

- y 是预测值
- x 是输入特征
- w 是权重参数（斜率）
- b 是偏置参数（截距）

损失函数

使用均方误差（MSE）作为损失函数：

$$MSE = (1/n) \cdot \sum (y_i - (w \cdot x_i + b))^2$$

代码实现：

```
func Mse(b, w float64, points [][]float64) float64 {
    totalError := 0.0
    for _, p := range points {
        x, y := p[0], p[1]
        totalError += math.Pow(y - (w*x+b), 2)
    }
    return totalError / float64(len(points))
}
```

梯度下降

梯度下降是一种优化算法，通过迭代更新模型参数来最小化损失函数。其核心思想是利用梯度信息来指导参数的更新方向。

梯度下降是一种优化算法，通过计算损失函数对参数的梯度（导数）来更新参数，使损失函数最小化。参数更新公式：

$$w = w - lr \cdot \partial(MSE) / \partial w$$
$$b = b - lr \cdot \partial(MSE) / \partial b$$

其中 lr 是学习率（步长），控制参数更新的幅度。

梯度计算与参数更新的代码实现：

```
func StepGradient(b, w float64, points [][]float64, lr float64) (float64, float64) {
    bGrad, wGrad := 0.0, 0.0 // 初始化梯度
    M := float64(len(points)) // 样本数量

    // 计算梯度
    for _, p := range points {
        x, y := p[0], p[1]
        err := w*x + b - y // 预测值与真实值的误差
        bGrad += (2 / M) * err // b的梯度
        wGrad += (2 / M) * x * err // w的梯度
    }

    // 更新参数
    return b - lr*bGrad, w - lr*wGrad
}
```

📌 梯度下降工作原理

每次迭代中，算法计算当前参数下的梯度，然后沿着梯度的反方向更新参数。通过不断迭代，参数将逐渐接近最优值，使损失函数达到最小值。

可视化部分

绘制元素

坐标系与网格

通过`drawGrid()`和`drawAxis()`函数实现：

- 绘制x轴和y轴，带箭头标识方向
- 添加网格线，便于观察数据位置
- 标注坐标轴刻度和单位

数据点与直线

通过`drawPoints()`和`drawLine()`函数实现：

- 绘制样本数据点（蓝色）
- 绘制真实直线（绿色）
- 绘制当前拟合直线（橙色）

状态信息展示

通过多个辅助函数展示训练状态：

```
// 绘制图例
func drawLegend(screen *ebiten.Image) { ... }

// 绘制统计信息
func drawStats(screen *ebiten.Image) { ... }

// 绘制进度条
func drawProgressBar(screen *ebiten.Image) { ... }
```

展示的关键信息包括：

- 当前迭代次数和总迭代次数
- 当前损失值（MSE）
- 当前模型参数 w 和 b
- 训练进度条
- 真实直线和拟合直线的方程

更新控制

Update()方法控制训练节奏：

```
func (g *Game) Update() error {
    // 检查是否达到更新间隔
    now := time.Now()
    if now.Sub(lastUpdate) >= time.Duration(updateInterval*float64(time.Second)) {
        b, w = StepGradient(b, w, data, lr) // 执行一次梯度下降
        step++                               // 迭代次数加1
        lastUpdate = now                     // 更新时间戳
    }
    return nil
}
```

通过时间控制确保更新频率稳定，避免过快更新导致可视化不流畅。

参数说明

关键参数

data	data	data	data
------	------	------	------

参数	含义	默认值	调整建议
dataSize	样本数据点数量	2000	数据越多，拟合越稳定，但计算量越大
lr	学习率	0.0005	过大会导致不收敛，过小会导致收敛慢
numIterations	总迭代次数	200000	足够大以确保收敛，可根据损失变化调整
Sigma	噪声标准差	1.548564	值越大，数据越分散，拟合难度越大
tw, tb	真实权重和偏置	1.72212862, 2.65145218	定义生成数据的真实线性关系